# Design and Implementation of a Database System for Lethbridge JonesAuto

**CPSC 3660 – Winter 2026**

Lance Tapang (001246536)

Eugene Gu (001233809)

Ryan Luu (001250767)

Austin Savoie (001252584)

Sofiah Plarisan (001251868)

## 1 Introduction

Given the circumstances of JonesAuto and their growing business, a means for more efficient data management became very apparent. Lethbridge JonesAuto, a used-car dealership relies on a manual paper-based system to manage operations. While the use of paper may be effective for a smaller scale business, it has become inefficient, error-prone, and difficult to maintain as the volume of transactions increases. The purpose of this project is to design and implement a database management system (DBMS) that replaces the existing manual system. Creating it such that it not only simulates the current business procedures but by digitizing the business operations you are thereby enhancing it. By emulating the procedures JonesAutos already have via digitizing business operations, better business decisions can be made and visualizing numbers for say accounting purposes will be far easier.

## 2 System Overview and Objectives

The primary goal of the system is to centralize and manage all business data related to all aspects of JonesAuto. Since we don't have to have a paper form we can further separate all of the data that had to be done manually previously which as seen, leads to redundancy. In addition to the ability to separate entries required per form, we also now have the ability to create further functionality for the data. Using a database model, data retrieval through reports and queries can lead to excellent business decisions. This idea of queries will be touched up on later on, however, it is important to note when determining the objectives that need to be completed.

## 3 Database Design

**Conceptual Design.** The conceptual design of the database is based on an analysis of the business processes described in the project requirements. The main entities identified include Customer, Employee, Car, Purchase, Sale, Warranty, Payment, and Repair. These entities represent the core components of the dealership's operations. Relationships between these entities were established to reflect real-world interactions. For example, a customer can be associated with multiple sales, while each sale involves exactly one car. Similarly, each car is acquired through a purchase transaction and may have multiple associated repair records. Warranties are linked to sales, and payments are linked to customers. Knowing this we should prepare a one-to-many relationship between said entities, again, another attribute of having a digitized system as opposed to a manual system.

**Logical Design and Normalization.** The logical design translates the conceptual model into relational tables. Each entity is represented as a table, with attributes corresponding to columns and primary keys uniquely identifying each record. Given the

nature of primary keys, it guarantees data integrity. Foreign keys are used to enforce relationships between tables, making sure that each entity created in relationship with one another can never be broken. The database schema was normalized to the Third Normal Form (3NF) to eliminate redundancy and ensure data integrity. For instance, repair details are stored in a separate table rather than being embedded within the purchase table, allowing multiple repair records per vehicle without duplication. Similarly, customer employment history is stored in a separate relation to support multiple entries per customer.

**UML Diagram.** The UML diagram visually represents the overall structure of the system and the relationships between entities defined in the conceptual and logical design phases. It provides a clear overview of how data flows within the dealership database and how different components interact with one another. Each class in the diagram corresponds directly to a relational table in the database, with attributes representing table columns and identifiers corresponding to primary keys. The connections between classes illustrate the relationships enforced through foreign keys in the schema. The diagram primarily uses one-to-many relationships to model real-world interactions. For example, a single customer may be associated with multiple sales and payments, while each sale is linked to exactly one car. Similarly, a car can have multiple repair records, reflecting its service history over time. These relationships ensure that the database structure remains scalable and avoids redundancy.

Figure 1: UML diagram

**Legend (Crow's Foot Notation)**

| | |
|---|---|
| `\|` | One |
| `O` | Zero |
| `<` | Many |
| `\|---\|` | One-to-one |
| `\|---<` | One-to-many |
| `O---<` | Zero-to-many |
| `>---<` | Many-to-many |

Figure 1 (Continued): Note that the legend above explains the notation used in the diagram. A vertical bar (|) indicates a mandatory participation of exactly one instance meaning that whatever entity is on the "—" side must have an input before being able to create the entity it is connected to, while a circle ($O$) represents optional participation. The crow's foot symbol (<) denotes "many", meaning there can be multiple instances of said entity. By combining these symbols, the diagram conveys the cardinality of relationships between entities. For instance, a |—< connection indicates that one instance of an entity can be associated with many instances of another, which is the most common relationship pattern in this system.

## 4 Implementation

The database system was implemented using MySQL for data storage and PHP for the front-end interface. The system provides several input forms that allow users to enter and update data. Using GET and POST functions to interact between the front end and backend, it is possible to then have the website and the database interact. It was decided that all three forms be separated in order to create the one-to-many relationships. Due to the nature of the forms (i.e. Payments, Sales, Warranty, Purchase) having each entity separated was the best decision. As an example, since Sale has a standard format that must be filled out each and every time for a new buyer, it leads to a lot of redundancy. If a customer wants to buy multiple vehicles, then filling out the customer form multiple times instead of having one customer key and then creating multiple vehicles seems silly to say the least. The idea to separate and create each relationship manually also makes it far easier for expansion if JonesAuto were to expand into other business models further down the line. The relationships allow for easy expansion due to there decoupled nature. In addition to input forms, the system generates output reports that provide insights into business operations. These include inventory reports showing available vehicles, sales reports summarizing

transactions and profits, and customer payment histories that track financial behavior.

## 5 Query Design

To support business decision-making, a set of PHP-based database queries was developed. These queries are executed through a PHP backend using the MySQL interface to join relevant data and allowing for further logic.

**Example PHP Query.** The following PHP code demonstrates a multi-table query that retrieves payment information.

```
$result = $conn->query("
    SELECT p.payment_id, p.customer_id, c.first_name, c.
       last_name,
           p.sale_id, s.sale_price, p.amount, p.payment_date,
           p.due_date, p.paid_date, p.days_late
    FROM payment p
    JOIN customer c ON p.customer_id = c.customer_id
    JOIN sale s ON p.sale_id = s.sale_id
    ORDER BY p.payment_date DESC
");
```

This PHP query uses the MySQL ‘query()‘ function to execute a multi-table JOIN. The result set combines payment records with customer and sale information, allowing the application to display enriched financial data. The results are ordered by payment date in descending order to prioritize recent transactions. This query demonstrates the use of **joins** to combine data from multiple related tables. The **JOIN** operations ensure that each payment record is enriched with corresponding customer and sale

information. Additionally, the **ORDER BY** clause sorts the results by payment date in descending order, allowing recent transactions to be viewed first. Such queries are essential for tracking payment activity, identifying late payments, and supporting financial reporting within the dealership system.

## 6 Inputs and Outputs

Here is an example of how purchase would work. To begin a vehicle form is always filled out for one purchase. Since any time a purchase is created there always has to be a vehicle you must submit a vehicle or select a specific vehicle to purchase. Another restriction put in place is that a salesperson must be selected. So here are the two inputs/outputs made before a purchase can actually be created.

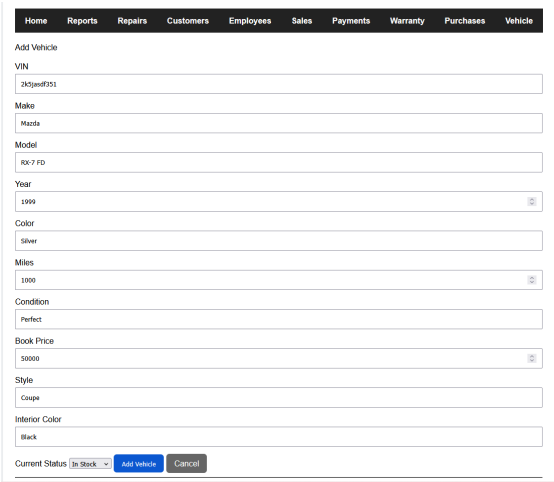


Figure 2: Input Vehicles

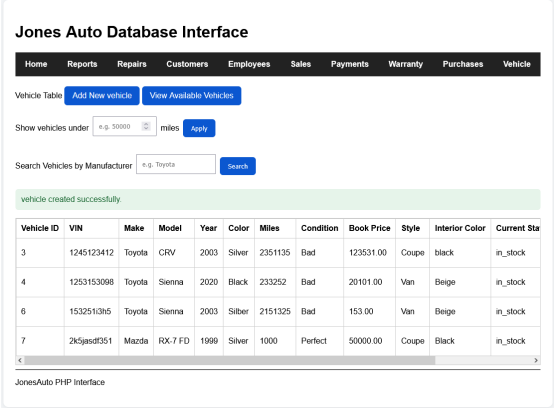


Figure 3: Output Vehicles

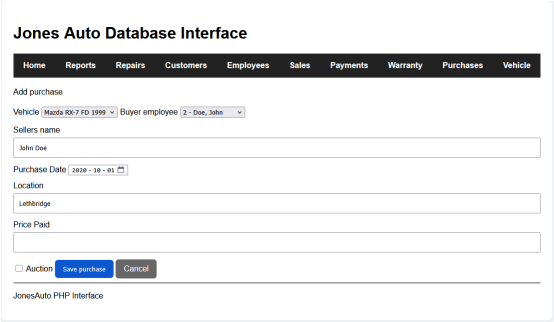


Figure 4: Purchase input

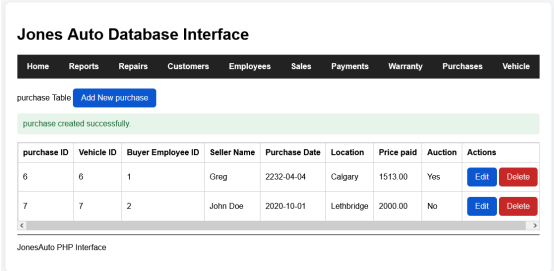


Figure 5: Purchase output

Given the purchase form is complete, a repair can be made on a per vehicle bases. You can now create as many repairs as needed.

Figure 6: Input Repairs

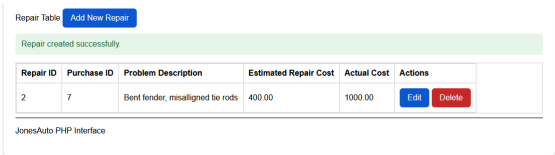


Figure 7: Outputs Repairs

This shows the full relation for purchase, creating the ability for employees to have multiple purchase records without having to fill out information for employee over and over again. Once an employee is inside of the database, a creation of said employee won't occur again.

## 7 Challenges and Limitations

One of the primary challenges encountered during the project was interpreting incomplete or ambiguous requirements. Without direct interaction with stakeholders, design decisions had to be made based on reasonable assumptions. Another challenge involved ensuring proper normalization while maintaining query efficiency. Balancing data integrity with performance required careful consideration of table structures and relationships. The system also has its limitations. For example, it does not currently include user authentication or role-based access control. Additionally, the user interface is functional but could be improved for better usability.

## 8 Assumptions

We made the assumption that this website would be used anyone affiliated with JonesAuto, meaning that the PHP/Website interactions is a full overview of what

the website would be. Roles and other traits proper websites need will/should be implemented later. Given the timeline, we made the assumption that it was to be made such that this was a full overview of said website. In other words, everything should be inserted manually to begin, once roles are defined further on and views are fully established then the website can actually be published. Another assumption made was that data overlap wouldn't occur. Basically saying that each and every single form is a new input and that the same form wouldn't be submitted twice. In the case where the exact same form is submitted twice over we wouldn't have a catcher for that.

## 9 What we learned

Austin - Overall, I gained an insight on the design and implementation of a simple database, along with my gained experience of PHP, and SQL. One thing I would have done differently is focusing more on the design of the database, so we would have spent less time working on removing redundancy and overall flow of the database would have been more efficient.

Ryan - My doing this project I feel like I gained a much deeper understanding of the process of planning, designing and then implementing a database. Furthermore, it also helped strengthen my skills in other tools such as PHP, CSS/HTML and SQL. The project really helped me visualize all the layers that go into a website/database and how they mesh together. One thing I would do different is making sure our starting ER diagram was built fundamentally strong. We had an issue where we had to add/make changes to entities on our base diagram. This made it confusing as we started coding and it was hard to keep track of things.

Lance - my biggest takeaway from working on this project was getting experience working with PHP. The project was helpful for gaining insight on the intricacies of creating databases, and the layers that make up a database. I also agree with Ryan, where I do think that if we had a more sound foundation in our ER diagram, our lives would have been easier at the start. I think another improvement we could make is implementing even more advanced queries to generate even more useful metrics for the reports section of the webpage.

Eugene - Throughout this project, I gained a much deeper understanding of full-stack application development, with a particular focus on the server-side components of a database-driven system. While the front-end provided an interface for user interaction, the majority of the complexity and learning came from designing and implementing the backend logic that connects the application to the database. Similar to Ryan I believe with a deeper understanding of ER diagrams would've led to a smoother development stage and less fuss around building the overall database.

Sofiah - This project provided valuable insight into the real-world applications of database systems. It began with a thorough analysis of the assignment requirements, similar to a professional setting, where understanding the specific needs of a client is crucial. Through this process, I learned how to translate a conceptual idea into a fully functional software application. Additionally, I gained practical experience in applying theoretical knowledge from lectures and assignments into coding using MySQL, PHP, and HTML. One area for improvement in future projects is ensuring that all team members are aligned from the beginning and progressing through the same project stages. In our case, multiple team members reflected that our ER diagram was not sufficiently solid. This issue stemmed from a lack of early coordination. Ensuring that the entire team finalizes the overall design phase before moving forward would help

prevent such issues and promote a more effective workflow.

## 10 Conclusion

The database system developed in this project successfully addresses the needs of Lethbridge JonesAuto by replacing manual processes with an efficient digital solution. The system ensures accurate data management, reduces redundancy, and provides valuable insights through queries and reports. Although the project represents a simplified version of a real-world application, it effectively demonstrates the principles and practices of database system design.